

Optimisation des cycles thermodynamiques de cogénération

Influence des conditions de température et de pression

Jéréemie **Prouteau**

PSI — N° 22852

2026

Sommaire

- 1 Introduction & Problématique
- 2 Modélisation théorique
- 3 Simulation numérique
- 4 Résultats paramétriques
- 5 Conclusion & Perspectives

Notations et conventions

Grandeurs thermodynamiques

Symbole	Grandeur	Unité
T	Température	K
P	Pression	Pa
v	Volume spécifique	m^3/kg
h	Enthalpie massique	J/kg
s	Entropie massique	J/(kg K)
W_{net}	Travail net massique	J/kg
Q_{in}	Chaleur reçue	J/kg
η	Rendement thermique	—
η_{Carnot}	Rendement de Carnot	—

Paramètres cycles

Symbole	Définition
T_{min}, T_1	Température froide
T_{max}, T_3	Température chaude
$\alpha_p = \alpha_p$	Rapport de pression
$\alpha_v = \alpha_v$	Rapport de volume
ε	Efficacité du régén.

Air — référence

$$r \approx 287 \text{ J/kg/K}$$

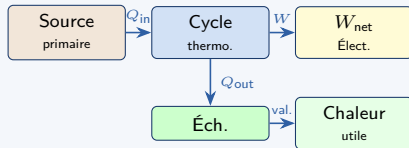
$$\gamma \approx 1,4$$

$$C_p \approx 1005 \text{ J/kg/K (300 K)}$$

La cogénération : définition et nécessité

Définition

Production **simultanée** d'électricité et de chaleur utile à partir d'une **unique source**, en valorisant Q_{out} .



Bilan comparé

Système	η_{global}
Centrale classique	33–40 %
Cogénération	80–90 %

Sur 100 kWh primaires :

Centrale : 35 kWh + 65 kWh perdus

Cogénération : 40 + 45 kWh valorisés

2^e principe

$$E_x = Q_{\text{out}} \left(1 - \frac{T_0}{T_{\text{cond}}} \right)$$

Rejeter Q_{out} détruit irréversiblement sa valeur : la cogénération l'exploite.

Problématique et cycles étudiés

Problématique

Comment les conditions de température et de pression ($T_{\min}$, $T_{\max}$, $P_{\min}$, $P_{\max}$) influencent-elles le rendement et la puissance spécifique des cycles thermodynamiques de cogénération ?

Brayton

Isentropique + Isobare

Cycle de référence
(turbine à gaz)

η dépend de α_p

Ericsson

Isotherme + Isobare

Turbomachine à
flux continu

$\eta = \eta_{\text{Carnot}}$ (avec rég.)

Stirling

Isotherme + Isochore

Moteur à pistons
alternatifs

$\eta = \eta_{\text{Carnot}}$ (avec rég.)

Les trois cycles sont étudiés avec **combustion externe** — le fluide de travail est l'air, chauffé via un échangeur. Ce mode est compatible avec toute source thermique industrielle.

Cadre thermodynamique

Conventions

- 1 kg de fluide (grandeurs massiques)
- Régime permanent : $\oint dU = 0$
- Fluide : Air

$$W_{\text{net}} = Q_{\text{in}} - Q_{\text{out}}$$

$$\eta = \frac{W_{\text{net}}}{Q_{\text{in}}} = 1 - \frac{Q_{\text{out}}}{Q_{\text{in}}}$$

Borne de Carnot

$$\eta \leq \eta_{\text{Carnot}} = 1 - \frac{T_{\text{min}}}{T_{\text{max}}}$$

Transformations clés — gaz parfait

Isobare	$\Delta s = C_p \ln(T_f/T_i)$
Isochore	$\Delta s = C_v \ln(T_f/T_i)$
Isotherme	$W = rT \ln(v_f/v_i)$
Isentropique	$T_f/T_i = (P_f/P_i)^{(\gamma-1)/\gamma}$

Diagramme T-s

Chaleur $Q = \int T ds \Rightarrow$ aire sous la courbe
Travail net = aire du cycle fermé

Démonstrations complètes : Annexes
A1–A3

Cycle de Brayton

Description des transformations

Transformations du cycle

	Transformation	Cste
1 → 2	Compression isentropique	s
2 → 3	Chauffage Isobare *	$P_{\max}$
3 → 4	Détente isentropique	s
4 → 1	Refroidissement isobare*	$P_{\min}$

* via le régénérateur

Points remarquables

T_1 : entrée compresseur (temp. froide)

T_3 : sortie chambre de combustion (temp. chaude)

Relation isentropique (Annexe A3) :

$$\frac{T_2}{T_1} = \frac{T_3}{T_4} = \alpha_p^{(\gamma-1)/\gamma}$$

Note : Les diagrammes comparatifs de tous les cycles sont regroupés à la fin de cette partie.

Cycle d'Ericsson (1/2)

Transformations et travail net

Transformations du cycle

	Transformation	Cste
1 → 2	Compression isotherme	$T_{\min}$
2 → 3	Chauffage isobare*	$P_{\max}$
3 → 4	Détente isotherme	$T_{\max}$
4 → 1	Refroidissement isobare*	$P_{\min}$

* via le régénérateur

Sur l'isotherme T (gaz parfait,
 $Pv = rT$) :

$$W = rT \ln(P_i/P_f)$$

$$W_{12} = -rT_{\min} \ln \alpha_p < 0$$

$$W_{34} = +rT_{\max} \ln \alpha_p > 0$$

Travail net

$$W_{\text{net}} = r(T_{\max} - T_{\min}) \ln \alpha_p$$

Cycle d'Ericsson (2/2)

Propriété clé

Propriété clé

$W_{\text{net}} \propto \ln \alpha_p$: croissance monotone.

W_{net} **augmente indéfiniment** avec le rapport de pression $\alpha_p = P_{\text{max}}/P_{\text{min}}$

Et cela **sans modifier le théorème de Carnot** ($\eta = 1 - T_{\text{min}}/T_{\text{max}}$).

⇒ **Séparation totale** entre optimisation de la puissance et du rendement.

Note : Les diagrammes de tous les cycles sont regroupés en fin de section.

Cycle de Stirling (1/2)

Description des transformations

Transformations du cycle

	Transformation	Cste
1 → 2	Compression isotherme	$T_{\min}$
2 → 3	Chauffage isochore*	$V_{\min}$
3 → 4	Détente isotherme	$T_{\max}$
4 → 1	Refroidissement isochore*	$V_{\max}$

* via le régénérateur

Propriété des isochores

Sur un isochore ($dV = 0$) :

$\delta W = P dV = 0$, **pas de travail.**

$$Q_{23} = C_v(T_{\max} - T_{\min}) > 0$$

$$Q_{41} = C_v(T_{\min} - T_{\max}) < 0$$

Le régénérateur stocke Q_{23} puis le restitue lors de Q_{41} : aucun échange extérieur de chaleur sur ces étapes.

Cycle de Stirling (2/2)

Calcul du travail net

Calcul du travail net

Seules les isothermes fournissent du travail. (Pour un gaz parfait : $dU = 0$ sur une isotherme) :

Détente $3 \rightarrow 4$ à $T_{\max}$:

$$W_{34} = rT_{\max} \ln(\alpha_v) > 0$$

Compression $1 \rightarrow 2$ à $T_{\min}$:

$$W_{12} = rT_{\min} \ln(1/\alpha_v) < 0$$

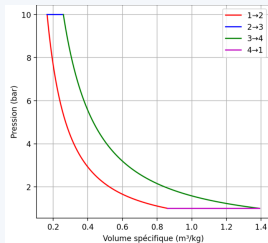
Travail net total

$$W_{\text{net}} = r(T_{\max} - T_{\min}) \ln \alpha_v$$

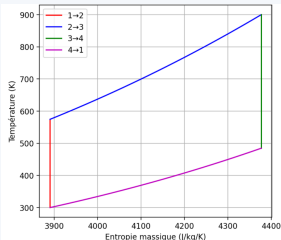
Diagrammes P-V et T-s des trois cycles

$T_1 = 300\text{ K}$, $T_{chaud} = 1100\text{ K}$, $P_{\min} = 2\text{ bar}$, $P_{\max} = 20\text{ bar}$, $r_v = 3,3$ (Stirling)

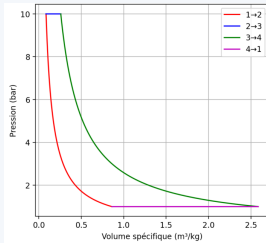
Brayton



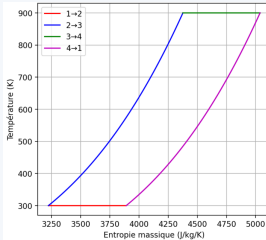
P-V



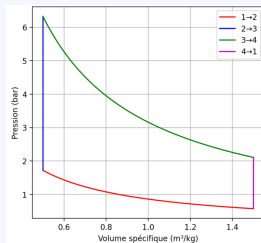
Ericsson



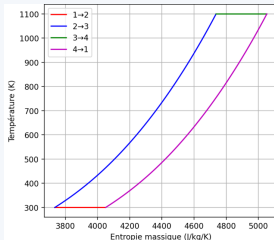
P-V



Stirling



P-V



Gaz parfait face au gaz réel : hypothèse

C_p de l'air n'est pas constant avec la température

Hypothèse gaz parfait

Le modèle gaz parfait suppose :

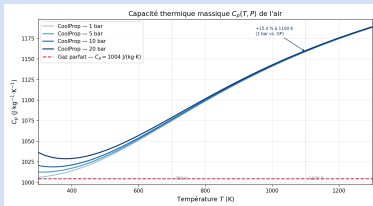
$$C_p = \text{cste}, C_v = \text{cste}, \gamma = \text{cste}$$

Valeurs de référence :

$$C_p \approx 1005 \text{ J/kg/K à } 300 \text{ K, } 1 \text{ bar}$$

Valables **uniquement** aux faibles écarts de température.

C_p de l'air en fonction de T



$C_p \text{ (J/kg/K) en fonction de } T \text{ (K) — CoolProp, 1 bar}$

Constat (mesures CoolProp)

Pour les cycles à haute température ($T_{\text{max}} \approx 1100 \text{ K}$), C_p augmente de +13,5 % entre 300 K et 1100 K.

Le modèle idéal **sous-estime** Q_{in} et **surestime** η .

Gaz parfait face au gaz réel : impact quantitatif

Écart mesuré sur C_p et conséquence sur le rendement simulé

Variation mesurée de C_p pour l'air

Condition	C_p (J/kg/K)	Écart
300 K, 1 bar	1 005	référence
700 K, 1 bar	1 075	+7 %
1100 K, 1 bar	1 141	+13,5 %
1100 K, 20 bar	1 148	+14,2 %

Écart résultant sur η : jusqu'à $-3,1$ points.

Conclusion

Le modèle gaz parfait **sous-estime** Q_{in} et **surestime** η pour $T_{max} > 700$ K.

CoolProp recalcule $h(T, P)$ et $s(T, P)$ avec des équations d'état précises à $< 0,1$ % (cf. Annexe A2).

Remarque : effet de P

L'effet pression (+0,7 % de 1 à 20 bar à 1100 K) est négligeable devant l'effet de température. CoolProp quantifie les deux.

Gaz parfait face à CoolProp — Impact numérique (1/2)

Comparaison pour le cycle d'Ericsson ($T_{\max} = 1100 \text{ K}$)

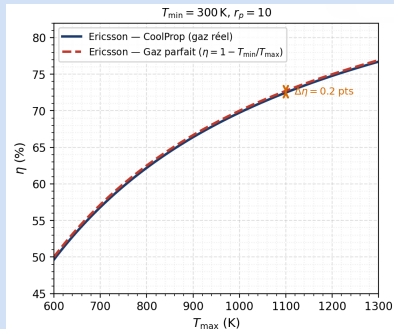
Comparaison quantitative

Grandeur	Gaz parfait	CoolProp
Q_{in} (kJ/kg)	232	252
W_{net} (kJ/kg)	169	175
η (%)	72,7	69,6

Écart sur η : **-3,1 points**

Cause : C_p réel $>$ C_p gaz parfait $\Rightarrow$
énergie de chauffage sous-estimée.

Courbes η en fonction de $T_{\max}$



η (%) en fonction de $T_{\max}$: gaz parfait (tireté) et CoolProp (trait plein)

Gaz parfait face à CoolProp — Impact numérique (2/2)

Bilan sur l'utilisation du modèle

Même analyse pour le cycle de Brayton

Gaz Parfait = 48,2 %

CoolProp = **50,9 %**

Écart *inverse* : $C_p(T)$ devient plus grand à haute température, ce qui défavorise la compression et favorise la détente, entraînant un équilibre différent.

Utilisation de CoolProp

$h =$

```
PropsSI("H","T",T,"P",P,"Air")
```

$s =$

```
PropsSI("S","T",T,"P",P,"Air")
```

L'écart justifie le passage à une résolution numérique car l'approximation s'effondre à haute température :

à 900 K : -1,5 pt

à 1300 K : -4 pts.

Validation numérique du code (1/2)

Méthodologie et vérification

Double vérification systématique

Méth. 1 — entropique :

$$Q_{\text{in}} = T_{\text{max}} \cdot (s_4 - s_3)$$

Méth. 2 — intégration numérique

$$\int P dv$$

Trajectoire découpée en 200 points
(méthode des trapèzes) :

$$W = \sum_i P_i \Delta v_i$$

Critère d'acceptation numérique

Pour valider la consistance de notre
intégration pas-à-pas avec le bilan
macroscopique :

$$\frac{|W_{\int P dv} - W_{T\Delta s}|}{|W_{T\Delta s}|} < 1\%$$

+ Déductions physiques vérifiées :

$$\eta \in [0, 1] \text{ et } \eta \leq \eta_{\text{Carnot}}$$

Validation numérique du code (2/2)

Architecture logicielle et optimisation

Architecture logicielle

Modules de simulation core :

Ericsson.py / Brayton.py /
Stirling.py

Optimisation Python :

@lru_cache(maxsize=50000)
appliqué sur l'appel thermodynamique.
gain $\times 5$ en temps de calcul

Interface de restitution : app.py
(Interface Streamlit interactive) pour
faire varier l'étude paramétrique en
temps réel et extraire le CSV de trace.

Impact sur la complexité

Pour accéder à la cartographie et
tracer une courbe complète, des
boucles massives sont nécessaires :

$$50 \times 4 \times 200 = 40\,000$$

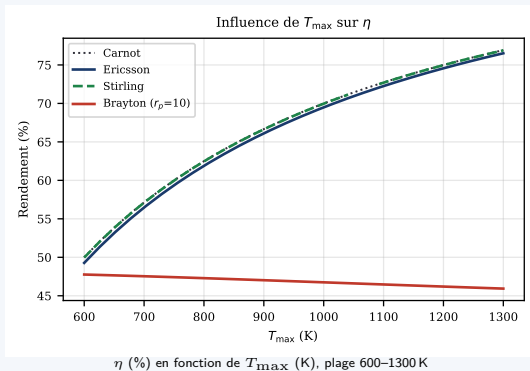
appels CoolProp unitaires par
paramètre simulé.

Le cache L1 applicatif évite les
réévaluations lourdes aux couples
(T, P) récurrents sur le réseau isoligné.

Site déployé publiquement : `tipecoge.streamlit.app`

Influence de $T_{\max}$ sur le rendement

$T_{\min} = 300 \text{ K}$ fixé, $\alpha_p = 10$



Ericsson & Stirling

$\eta = 1 - 300/T_{\max}$: croissance
monotone.

900 K	66,7 %
1100 K	72,7 %
1300 K	76,9 %

Brayton

η **constant** avec $T_{\max}$ (pour α_p fixé)

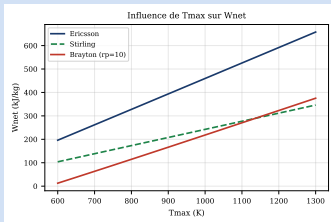
Levier principal

Maximiser $T_{\max}$ est le levier principal sur η .

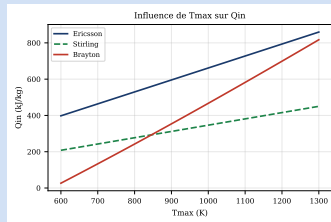
Influence de $T_{\max}$ sur W_{net} et Q_{in}

$T_{\min} = 300 \text{ K}$ fixé, $\alpha_p = 10$

$W_{\text{net}}(T_{\max})$



$Q_{\text{in}}(T_{\max})$



Lecture comparée

Ericsson & Stirling : W_{net} et Q_{in} croissent ensemble — η suit Carnot.

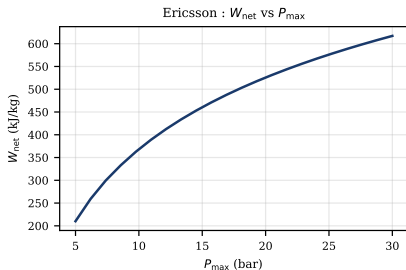
Brayton : W_{net} croît mais η reste constant (α_p fixé), Q_{in} croît proportionnellement.

Synthèse progressive des propriétés théoriques

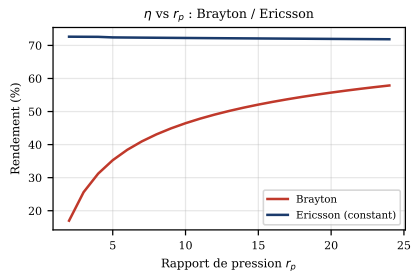
	Brayton	Ericsson	Stirling
η dépend de $T_{\max}$?	Non	Oui	Oui

Influence de $P_{\max}$ et α_p

Travail net selon $P_{\max}$



Rendement selon α_p (Brayton)



Mécanismes de pression

Le rendement de Brayton est déterminé par α_p .

Le travail net des trois cycles dépend fortement de la densité du gaz, donc de $P_{\max}$.

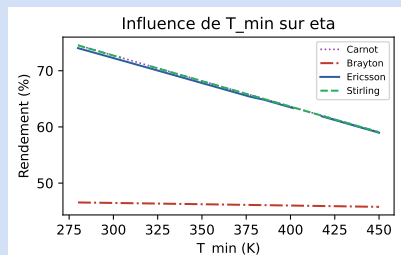
Synthèse progressive des propriétés théoriques

	Brayton	Ericsson	Stirling
η dépend de $T_{\max}$?	Non	Oui	Oui
η dépend de $P_{\max}$?	Oui (via α_p)	Non	Non
W_{net} dépend de $P_{\max}$?	Oui (optimum α_p^*)	Oui ($\ln \alpha_p$)	Non

Influence de $T_{\min}$ — Étude quantitative (1/2)

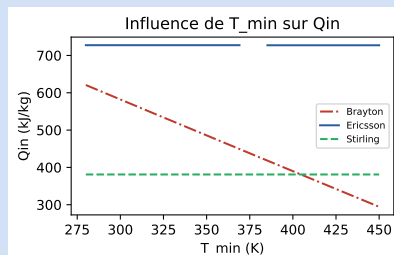
Impact thermique sur l'enveloppe de conception

η en fonction de $T_{\min}$



Brayton, Ericsson, Stirling, Carnot

Q_{in} en fonction de $T_{\min}$



Q_{in} (kJ/kg) : Brayton, Ericsson, Stirling

Influence de $T_{\min}$ — Bilan (2/2)

Le compromis de cogénération

Dilemme de la cogénération

$T_{\min}$ **bas** (ex : 300 K) :

$\eta_{\text{élec}}$ maximal, mais Q_{utile} est évacué à très basse température, ce qui n'est pas exploitable ni pour le réseau de chaleur ni pour un process industriel.

$T_{\min}$ **élevé** (ex : 380 K - 450 K) :

$\eta_{\text{élec}}$ réduit drastiquement, mais Q_{utile} devient hautement **valorisable**.

Rendement global du système

$$\eta_g = \frac{W_{\text{net}} + Q_{\text{utile}}}{Q_{\text{source}}}$$

Au bilan net réel, l'efficacité thermique η_g d'utilisation du combustible reste **maximale** même si l'efficacité électrique baisse.

⇒ Le choix de $T_{\min}$ doit être dicté par les coûts d'énergie respectifs et la nature des besoins thermiques et non en optimisant aveuglément la machine thermique isolée.

Synthèse progressive des propriétés théoriques

	Brayton	Ericsson	Stirling
η dépend de $T_{\max}$?	Non	Oui	Oui
η dépend de $P_{\max}$?	Oui (via α_p)	Non	Non
W_{net} dépend de $P_{\max}$?	Oui (optimum α_p^*)	Oui ($\ln \alpha_p$)	Non
Régénération nécessaire	Non	Isobare	Isochore
$\eta_{\text{théorique}}$	$< \eta_{\text{Carnot}}$	$= \eta_{\text{Carnot}}$	$= \eta_{\text{Carnot}}$

Ericsson et Stirling atteignent la borne de Carnot grâce à la régénération idéale ($\varepsilon = 1$). Pour Brayton, l'absence de régénérateur impose $\eta < \eta_{\text{Carnot}}$ quel que soit α_p : le rendement est piloté uniquement par le rapport de compression.

Les trois cycles : résultats CoolProp

Conditions identiques : $T_{\min} = 300 \text{ K}$, $T_{\max} = 1100 \text{ K}$, $r_p = 10$, $r_v = 3$ (Stirling)

	Brayton	Ericsson	Stirling
η simulé avec CoolProp (%)	$\approx 50,9$	$\approx 69,6$	$\approx 69,6$
W_{net} (kJ/kg)	≈ 283	≈ 349	≈ 231
Q_{in} (kJ/kg)	≈ 556	≈ 502	≈ 332

Brayton versus Ericsson

Écart de rendement : ≈ 19 points.

Cause unique : absence de régénérateur dans Brayton $\Rightarrow Q_{\text{in}}$ bien plus élevé (chauffage isobare depuis $T_2 \ll T_{\max}$).

Stirling

η identique à Ericsson, mais W_{net} plus faible :

le rapport de volume r_v limite le travail net, alors que Ericsson peut augmenter r_p librement.

Synthèse et réponse à la problématique

Ericsson & Stirling

$$\eta = 1 - \frac{T_{\min}}{T_{\max}}$$

Seul $T_{\max}$ pilote η .

$P_{\max}$ pilote W_{net} sans affecter η .

⇒ Séparation puissance/rendement.

Tableau de synthèse

	B	E	S
$\eta = \eta_{\text{Carnot}}$	×	✓	✓
W_{net} via P	opt.	✓	×
Grandes MW	✓	✓	×

Brayton

$$\eta = 1 - \alpha_p^{(1-\gamma)/\gamma}$$

Seul α_p pilote η .

T_3 (sortie chambre) n'influe pas sur η
(se simplifie dans $\frac{W_{\text{net}}}{Q_{\text{in}}}$).

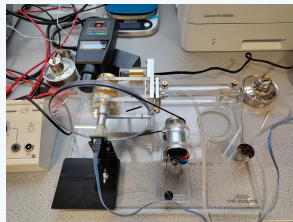
W_{net} passe par un maximum en α_p^* .

Bilan numérique

Modélisation CoolProp (gaz réel),
validation croisée $T\Delta s / \int P dv$ (écart
< 1 %), étude paramétrique complète.

Validation expérimentale — Montage

Moteur Jeulin 204081 (3B Scientific) — ExAO Foxy



Validation expérimentale — Caractéristiques

Détails du dispositif expérimental

Dispositif expérimental

- Moteur Stirling α transparent (cylindres en verre thermorésistant)
- Source chaude : réchaud à alcool
- Capteur PV Stirling Jeulin branché sur interface ExAO Foxy
- Acquisition numérique en temps réel de la pression $P(t)$ et du volume $V(t)$
- Thermocouple de type K pour T_{chaud} et T_{froid}

Paramètres du moteur

Paramètre	Valeur
$V_{\min} / V_{\max}$	32 / 44 cm ³
$\alpha_v = V_{\max}/V_{\min}$	1,375
Puissance nominale	1,5 W
Vitesse sans charge	$\approx 1\,000$ tr/min

Diagramme P-V expérimental du cycle de Stirling (1/2)

Mise en évidence par acquisition ExAO — Visuel brut

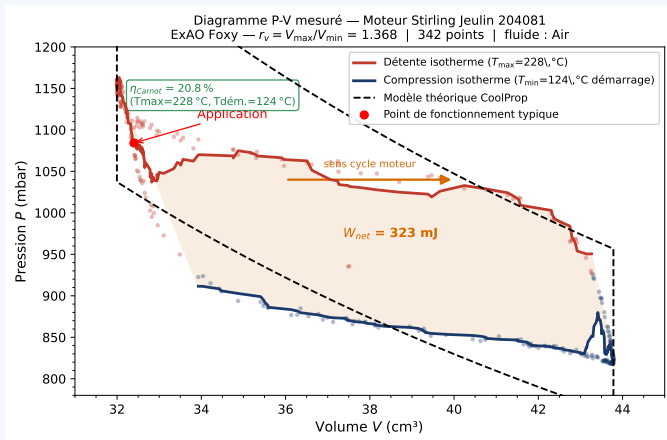


Diagramme de Clapeyron mesuré (P en mbar, V en cm^3).
 $T_{\text{démarrage}} = 124^\circ\text{C}$, $T_{\max} = 228^\circ\text{C}$

Diagramme P-V expérimental du cycle de Stirling (2/2)

Analyse des résultats du graphe

Lecture du diagramme

Branche haute (courbe rouge) :

Phase de détente s'approchant de l'isotherme à chaude $T_{\max} = 228^{\circ}\text{C}$.

Branche basse (courbe bleue) :

Phase de compression à $T_{\min}$.

Intégration de l'aire

L'aire algébrique interne au cycle correspond au travail net :

$$W_{\text{net}} = \oint P dV$$

Valeurs mesurées

$$\alpha_v^{\text{exp}} \quad 1,368$$

$$\Delta P_{\max} \quad \approx 246 \text{ mbar}$$

$$W_{\text{net}} \quad \mathbf{323 \text{ mJ /cycle}}$$

Plafond théorique limitant

En modèle de Carnot, avec les extrêmes de la machine :

$$\eta_{\text{Carnot}} = 1 - \frac{397 \text{ K}}{501 \text{ K}} \approx \mathbf{20,8 \%}$$

Confrontation modèle – mesure (1/2)

Identification de l'efficacité du régénérateur ε

Modèle numérique actualisé

Avec $T_{\min} = 397 \text{ K}$, $T_{\max} = 501 \text{ K}$ et le rapport mesuré $\alpha_v = 1,368$:

On évalue la chaleur effectivement reçue par le fluide en tenant compte de l'imperfection ε du régénérateur :

$$Q_{\text{in,eff}} = Q_{34} + (1 - \varepsilon) Q_{23,\text{rég}}$$

$$\Rightarrow \eta_{\text{mod}} = \frac{W_{\text{net}}}{Q_{\text{in,eff}}} < \eta_{\text{Carnot}}$$

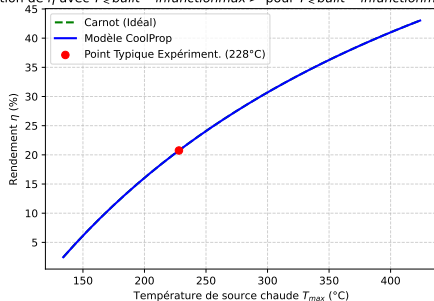
Bilan numérique vs de Carnot

Grandeur limitante	Valeur
α_v^{exp} expérimental	1,368
W_{net} réel exp (ExAO)	323 mJ/ct
η_{Carnot} plafond de Carnot	20,8 %
η_{mod} ($\varepsilon = 0,85$)	$\approx 12 \%$

Confrontation modèle – mesure (2/2)

Synthèse et Origine des dégradations de rendement

Représentation de η avec $T_{<built>-inf}$ fonction $T_{<built>-inf}$ pour $T_{<built>-inf}$ fonction $T_{<built>-inf}$



η en fonction de T_{max} — modèle vs Carnot

Conclusion expérimentale

Limitation architecturale : le modèle révèle qu'un si faible $\alpha_v \approx 1,37$ bridera immédiatement les capacités du moteur (puisque $W_{net} \propto \ln \alpha_v$).

Chute de rendement :

L'écart immense entre η_{Carnot} (20 %) et η_{mod} (12 %) s'explique analytiquement et intègre naturellement les irréversibilités thermiques : la **régénération non idéale** ($\varepsilon \approx 0,85$) et un échange thermodynamique continu.

Outil fondamental pour toutes les démonstrations : Origine

Combinaison du 1^{er} et du 2^e principe pour une transformation **réversible** :

1^{er} principe : $dU = \delta Q + \delta W = T ds - P dv$

$$T ds = dU + P dv = dH - v dP$$

C'est la **relation de Gibbs**, valable pour tout état d'équilibre d'un corps pur, même irréversiblement atteint (car s , U , H sont des fonctions d'état).

Applications immédiates pour la simulation

Isobare ($dP = 0$) :

Isochore ($dv = 0$) :

Isentropique ($ds = 0$) :

Isotherme ($dT = 0$, gaz parfait) :

$$dU = 0 \Rightarrow \delta Q = \delta W = P dv$$

A3 — Relation isentropique (Brayton)

Énoncé : Montrer que $T_2/T_1 = \alpha_p^{(\gamma-1)/\gamma}$ pour une compression adiabatique réversible.

Démonstration condensée

Relation de Gibbs en isentropique ($ds = 0$) avec l'équation du gaz parfait ($Pv = rT$) :

$$T ds = dH - v dP = C_p dT - \frac{rT}{P} dP = 0 \quad \Rightarrow \quad \frac{dT}{T} = \frac{r}{C_p} \frac{dP}{P}$$

On sait que $r = C_p - C_v$ d'après la relation de Mayer, donc :

$$\frac{r}{C_p} = 1 - \frac{C_v}{C_p} = 1 - \frac{1}{\gamma} = \frac{\gamma - 1}{\gamma}$$

En intégrant de 1 à 2 ($\int dx/x = \ln x$) :

$$\ln\left(\frac{T_2}{T_1}\right) = \frac{\gamma-1}{\gamma} \ln\left(\frac{P_2}{P_1}\right) \Rightarrow \boxed{\frac{T_2}{T_1} = \alpha_p^{\frac{\gamma-1}{\gamma}}}$$

Démonstration complète

Poser $x = \alpha_p^{(\gamma-1)/\gamma} : T_2 = T_1 x, T_4 = T_3/x.$

$$\eta = 1 - \frac{T_4 - T_1}{T_3 - T_2} = 1 - \frac{T_3/x - T_1}{T_3 - xT_1}$$

Factoriser par $1/x$ au numérateur :

$$= 1 - \frac{(T_3 - xT_1)/x}{T_3 - xT_1} = 1 - \frac{1}{x}$$

$$\eta_{\text{Brayton}} = 1 - \alpha_p^{(1-\gamma)/\gamma}$$

Propriété : le facteur $(T_3 - xT_1)$ se simplifie exactement — η est indépendant de T_3 (température en sortie de chambre de combustion).

A5 — Rapport de pression optimal α_p^* (Brayton)

Maximisation de W_{net}

En posant $x = \alpha_p^{(\gamma-1)/\gamma}$ et $\tau = \frac{T_3}{T_1}$ (rapport des températures source/puits) :

$$W_{\text{net}} = C_p T_1 \left(\tau - \frac{\tau}{x} - x + 1 \right)$$

Condition nécessaire d'extremum (dérivée première nulle) :

$$\frac{dW_{\text{net}}}{dx} = C_p T_1 \left(\frac{\tau}{x^2} - 1 \right) = 0 \Rightarrow x = \sqrt{\tau}$$

Condition suffisante (dérivée seconde négative) : $d^2W_{\text{net}}/dx^2 = -2C_p T_1 \tau / x^3 < 0$, c'est bien un **maximum**.

On trouve en isolant α_p :

$$\alpha_p^* = (\tau)^{\frac{\gamma}{2(\gamma-1)}}$$

Application : $T_3/T_1 = 1100/300 \approx 3,67$, $\gamma = 1,4 \Rightarrow \alpha_p^* \approx 8,6$.

A6 — Entropie isobare (Ericsson)

Énoncé : Calculer Δs_{23} sur l'isobare 2→3 de $T_{\min}$ à $T_{\max}$ et montrer que $|\Delta s_{23}| = |\Delta s_{41}|$.

Démonstration

Relation de Gibbs + isobare ($dP = 0$) :

$$T ds = dH = C_p dT \Rightarrow ds = \frac{C_p}{T} dT$$

Intégration de $T_{\min}$ à $T_{\max}$:

$$\Delta s_{23} = C_p \ln \frac{T_{\max}}{T_{\min}} > 0$$

Pour l'isobare 4→1 (de $T_{\max}$ à $T_{\min}$) :

$$\Delta s_{41} = C_p \ln \frac{T_{\min}}{T_{\max}} = -\Delta s_{23}$$

$$\Rightarrow |Q_{23}| = C_p(T_{\max} - T_{\min}) = |Q_{41}|$$

Avec $C_p(T)$ variable (CoolProp) : même conclusion, les deux isobares couvrant

A7 — Travail net Ericsson

Démonstration

Détente 3→4 à $T_{\max}$, de $P_{\max}$ à $P_{\min}$ (gaz parfait : $Pv = rT_{\max}$) :

$$W_{34} = \int_{v_3}^{v_4} P dv = rT_{\max} \ln \frac{v_4}{v_3} = rT_{\max} \ln \frac{P_3}{P_4} = rT_{\max} \ln \alpha_p$$

Compression 1→2 à $T_{\min}$, de $P_{\min}$ à $P_{\max}$:

$$W_{12} = rT_{\min} \ln \frac{v_2}{v_1} = rT_{\min} \ln \frac{P_{\min}}{P_{\max}} = -rT_{\min} \ln \alpha_p$$

Les isobares $2 \rightarrow 3$ et $4 \rightarrow 1$ sont internes (régénération) : W_{23} et W_{41} se compensent globalement.

$$W_{\text{net}} = r(T_{\text{max}} - T_{\text{min}}) \ln \alpha_p$$

A8 — $\eta_{\text{Ericsson}} = \eta_{\text{Carnot}}$

Étape 1 — Régénération exacte (cf. A6)

$\Delta_{s23} = -\Delta_{s41} \Rightarrow |Q_{23}| = |Q_{41}|$: avec $\varepsilon = 1$, les isobares ne font **aucun échange extérieur**.

Étape 2 — Bilan entropique du cycle

Seuls échanges extérieurs : $Q_{\text{in}} = T_{\text{max}}(s_4 - s_3)$, $Q_{\text{out}} = T_{\text{min}}(s_1 - s_2)$.

$\oint ds = 0$ et étape 1 $\Rightarrow (s_3 - s_2) = -(s_1 - s_4)$, donc :

$$(s_2 - s_1) + (s_4 - s_3) = 0 \Rightarrow |s_4 - s_3| = |s_2 - s_1| \triangleq |\Delta s|$$

Étape 3 — Rendement

$$\eta_{\text{Ericsson}} = 1 - \frac{Q_{\text{out}}}{Q_{\text{in}}} = 1 - \frac{T_{\text{min}} |\Delta s|}{T_{\text{max}} |\Delta s|} = 1 - \frac{T_{\text{min}}}{T_{\text{max}}} = \eta_{\text{Carnot}}$$

A9 — Démonstration : $\eta_{\text{Stirling}} = \eta_{\text{Carnot}}$

Étape 1 — Régénération isochore

Sur un isochore ($dv = 0$, $\delta W = 0$) : $\delta Q = C_v dT$. **2→3** : $Q_{23} = C_v(T_{\text{max}} - T_{\text{min}}) > 0$
4→1 : $|Q_{41}| = C_v(T_{\text{max}} - T_{\text{min}}) = Q_{23}$ Avec $\varepsilon = 1$: aucun échange extérieur.

Étape 2 — Variations d'entropie des isochores

$$\Delta s_{23} = C_v \ln \frac{T_{\text{max}}}{T_{\text{min}}}, \quad \Delta s_{41} = C_v \ln \frac{T_{\text{min}}}{T_{\text{max}}} = -\Delta s_{23}$$

Même bilan entropique qu'Ericsson : $|s_4 - s_3| = |s_2 - s_1| \triangleq |\Delta s|$

Étape 3 — Rendement

Seuls échanges extérieurs : isothermes.

$$\eta_{\text{Stirling}} = 1 - \frac{T_{\text{min}} |\Delta s|}{T_{\text{max}} |\Delta s|} = 1 - \frac{T_{\text{min}}}{T_{\text{max}}} = \eta_{\text{Carnot}}$$

A10 — Régénérateur imparfait ($\varepsilon < 1$)

Modèle

$\varepsilon = Q_{\text{transmis}}/Q_{\text{idéal}} \in [0, 1]$. La fraction $(1 - \varepsilon)$ non transmise est fournie par la source externe :

$$Q_{\text{in,eff}} = Q_{34} + (1 - \varepsilon) C_p (T_{\text{max}} - T_{\text{min}})$$

Rendement effectif

$$\eta_{\varepsilon} = \frac{r(T_{\max} - T_{\min}) \ln \alpha_p}{Q_{34} + (1 - \varepsilon)C_p \Delta T} < \eta_{\text{Carnot}}$$

$$\varepsilon = 1 : \eta = \eta_{\text{Carnot}}$$

$\varepsilon = 0,85$: perte $\approx 5-10$ pts

$$\varepsilon = 0 : \eta \ll \eta_{\text{Carnot}}$$

Interprétation

Le régénérateur réduit Q_{in} sans changer W_{net} .

Sans lui (Brayton) : $Q_{in} = Q_{34} + Q_{23}$,
d'où l'écart de ≈ 19 pts avec Ericsson à
conditions identiques.

Paramètre critique : la mesure de ε est l'objectif principal de la partie expérimentale.

Définition formelle thermodynamique

L'exergie d'une chaleur Q disponible à T dans un environnement à T_0 est le travail utile **maximal** extractible, en imaginant une machine de Carnot réversible placée entre T et T_0 :

$$E_x = Q \left(1 - \frac{T_0}{T} \right)$$

Vapeur haute enthalpie 200°C : $E_x \approx 38 \% \cdot Q$

Eau de chauffage 70°C : $E_x \approx 15 \% \cdot Q$

Rejet condensateur 47°C : $E_x \approx 8\% \cdot Q$

A11 — Exergie et cogénération (2/2)

Outil d'évaluation à la faveur de la cogénération

Destruction d'exergie au condenseur pur

Rejeter brutalement Q_{out} à T_0 crée irrémédiablement de l'entropie :

$$\dot{S}_{\text{irr}} = Q_{\text{out}} \left(\frac{1}{T_0} - \frac{1}{T_{\text{cond}}} \right) \geq 0$$

Motivation thermo-conceptuelle pour la cogénération

Dans une centrale classique, Q_{out} est rejeté à $T_{\text{cond}} \approx T_0$: son exergie résiduelle est quasi nulle ($\approx 8\%$) et **détruite irréversiblement** par transfert thermique hors l'équilibre vers le milieu ambiant.

En cogénération certifiée, Q_{out} est valorisé réfléchi avant d'atteindre le point mort et limitant T_0 : on récupère d'abord l'essentiel du coût de pompage sous forme d'exergie *réutilisable*.

C'est la preuve fondamentale, au sens du 2^e principe, en faveur d'une telle

A12 — Code : Ericsson.py

```

from CoolProp.CoolProp import PropsSI as CP
from functools import lru_cache
import numpy as np

fluid = "Air"

@lru_cache(maxsize=50000)
def CP_cached(out, i1, v1, i2, v2, fl):
    return CP(out, i1, v1, i2, v2, fl)

def get_s(T, P): return CP_cached("S","T",T,"P",P,fluid)
def get_h(T, P): return CP_cached("H","T",T,"P",P,fluid)

def cycle_ericsson(T_min=300, T_max=1100, P_min=2e5, P_max=20e5):
    s1, s2 = get_s(T_min,P_min), get_s(T_min,P_max)
    s3, s4 = get_s(T_max,P_max), get_s(T_max,P_min)

    Q_in  = T_max*(s4-s3)    # chaleur recue (isotherme 3->4)
    Q_out = T_min*(s1-s2)    # chaleur cedee (isotherme 1->2)
    W_net = Q_in - Q_out
    eta   = W_net / Q_in

    # Validation croisee int(P dv) via trapeze sur 200 pts
    # Critere : |W_pdv - W_net| / |W_net| < 1 %
    return {"Q_in":Q_in, "Q_out":Q_out, "W_net":W_net, "eta":eta}
  
```

A13 — Code : Brayton.py

```
def cycle_brayton(T1=300, P1=2e5, pressure_ratio=10, T3=1100):
    P2 = P1 * pressure_ratio

    # Etat 2 : compression isentropique (s constant = s1)
    s1 = get_s(T1, P1)
    T2 = CP("T", "P", P2, "S", s1, fluid) # T tel que s(T,P2)=s1

    # Etat 3 : fin chauffage isobare
    s3 = get_s(T3, P2)

    # Etat 4 : detente isentropique (s constant = s3)
    T4 = CP("T", "P", P1, "S", s3, fluid)

    h1, h2 = get_h(T1, P1), get_h(T2, P2)
    h3, h4 = get_h(T3, P2), get_h(T4, P1)

    W_comp = h2-h1          # travail compresseur (recu)
    W_turb = h3-h4          # travail turbine (fourni)
    W_net = W_turb - W_comp
    Q_in = h3-h2

    return {"W_net":W_net, "Q_in":Q_in, "eta":W_net/Q_in,
           "T2":T2, "T4":T4}
```

A14 — Code : Stirling.py

```
def cycle_stirling(T_min=300, T_max=1100, V_min=0.3, V_max=1.0):
    # Pression via equation d'etat : rho = 1/v
    def get_P(T,V):
        return CP_cached("P", "T", T, "D", 1/V, fluid)

    P1 = get_P(T_min, V_max) # etat 1 : froid, grand vol.
    P2 = get_P(T_min, V_min) # etat 2 : froid, petit vol.
    P3 = get_P(T_max, V_min) # etat 3 : chaud, petit vol.
    P4 = get_P(T_max, V_max) # etat 4 : chaud, grand vol.

    s1,s2 = get_s(T_min,P1), get_s(T_min,P2)
    s3,s4 = get_s(T_max,P3), get_s(T_max,P4)

    Q_in  = T_max*(s4-s3) # isotherme 3->4
    Q_out = T_min*(s1-s2) # isotherme 1->2

    # Chaleur isochores (regeneration interne)
    Q_regen = get_h(T_max,P3) - get_h(T_min,P2)

    W_net = Q_in - Q_out
    return {"Q_in":Q_in, "Q_out":Q_out, "W_net":W_net,
            "Q_regen":Q_regen, "eta":W_net/Q_in,
            "eta_carnot":1-T_min/T_max}
```

A15 — app.py : boucle paramétrique

Simulation itérative avec `numpy.linspace` et stockage des résultats

Extrait — étude paramétrique (cycle d'Ericsson)

```
import numpy as np, pandas as pd
from src import Ericsson

for param_to_vary in params_to_vary:
    p_min, p_max = param_ranges[param_to_vary]
    param_values = np.linspace(p_min, p_max, n_points) # 5..50 pts
    results_list = []

    for val in param_values:
        # Remplace la variable, fixe les autres
        kwargs = dict(T_min=T_min_fixed, T_max=T_max_fixed,
                      P_min=P_min_fixed, P_max=P_max_fixed)
        kwargs[param_to_vary] = val

        if kwargs["T_max"] <= kwargs["T_min"]: continue
        if kwargs["P_max"] <= kwargs["P_min"]: continue

        res = Ericsson.cycle_ericsson(**kwargs, show_plot=False)
        ener = res["energetics"]
        results_list.append({
            param_to_vary: val,
            "Qin (kJ/kg)": ener["Q_in"] / 1000,
            "Wnet (kJ/kg)": ener["W_cycle"] / 1000,
            "Rendement (%)": ener["eta"] * 100,
        })

    all_results[param_to_vary] = pd.DataFrame(results_list)
```

A16 — app.py : graphique interactif

Superposition des courbes paramétriques avec Altair (create_overlay_chart)

Extrait — construction du graphique superposé

```
def create_overlay_chart(all_results, metric_col, y_title):
    dfs = []
    for param_name, df in all_results.items():
        df_copy = df.copy()
        df_copy["Point"] = df_copy.index
        df_copy["Parametre"] = param_name
        df_copy["Metrique"] = df_copy[metric_col]
        dfs.append(df_copy)
    combined = pd.concat(dfs, ignore_index=True)
    # Selection au survol
    nearest = alt.selection_point(nearest=True, on="pointerover",
                                  fields=["Point"], empty=False)

    line = alt.Chart(combined).mark_line(strokeWidth=2.5).encode(
        x=alt.X("Point:Q", title="Point de simulation"),
        y=alt.Y("Metrique:Q", title=y_title),
        color=alt.Color("Parametre:N", legend=alt.Legend(orient="bottom")),
    )
    points = line.mark_circle(size=120).encode(
        opacity=alt.condition(nearest, alt.value(1), alt.value(0)),
        tooltip=["Parametre:N", "Valeur affichee:Q", "Metrique:Q"],
    ).add_params(nearest)

    return alt.layer(line, points).interactive()

# Affichage dans Streamlit (un graphe par metrique)
st.altair_chart(create_overlay_chart(all_results, "Rendement (%)", "eta (%)",
                                     use_container_width=True))
```

A17 — $C_{pv} s_T.py(1/2)$

Comparatif du C_p de l'air

```

"""
Trace de  $C_p(T)$  pour l'air - CoolProp vs. gaz parfait
Sauvegarde : fig/Cp_vs_T.pdf et fig/Cp_vs_T.png
"""

import numpy as np
import matplotlib.pyplot as plt
from CoolProp.CoolProp import PropsSI

# __ Parametres __
T_min, T_max = 300, 1300          # plage de temperatures (K)
N = 300                          # points de discretisation
pressures_bar = [1, 5, 10, 20]   # pressions (bar)
Cp_gp = 1004.5                   # gaz parfait reference ISO (J/kg/K)

# __ Calcul CoolProp __
T = np.linspace(T_min, T_max, N)

fig, ax = plt.subplots(figsize=(10, 5.5))
colors = plt.cm.Blues(np.linspace(0.4, 1.0, len(pressures_bar)))

for P_bar, color in zip(pressures_bar, colors):
    P_Pa = P_bar * 1e5
    Cp = np.array([PropsSI("Cpmass", "T", Ti, "P", P_Pa, "Air") for Ti in T])
    ax.plot(T, Cp, color=color, lw=2.0, label=f"CoolProp - {P_bar} bar")

# Reference gaz parfait
ax.axhline(Cp_gp, color="crimson", lw=1.8, ls="--",
           label=f"Gaz parfait -  $C_p = \{Cp\_gp:.0f\}$  J/(kg/K)")

# __ Annotations temperatures d'interet __

```

A17 — $C_{p_{vST.py}}(2/2)$

Comparatif du C_p de l'air

```
# __ Mise en forme -----
ax.set_xlabel("Temperature  $T$  (K)", fontsize=12)
ax.set_ylabel(r"$C_p$ (J/kg$^{-1}$ / K$^{-1}$)", fontsize=12)
ax.set_title(r"Capacite thermique massique  $C_p(T,P)$  de l'air", fontsize=13)
ax.set_xlim(T_min, T_max)
ax.legend(fontsize=10, loc="upper left")
ax.grid(True, alpha=0.3)

# cart relatif a 1100 K pour annotation
P1_Pa = 1e5
Cp_1100_cool = PropsSI("Cpmass", "T", 1100, "P", P1_Pa, "Air")
ecart = (Cp_1100_cool - Cp_gp) / Cp_gp * 100
ax.annotate(f"+{ecart:.1f} % a 1100 K\n(1 bar vs. GP)",
            xy=(1100, Cp_1100_cool), xytext=(950, Cp_1100_cool + 20),
            fontsize=8, color=colors[-1],
            arrowprops=dict(arrowstyle="->", color=colors[-1], lw=1.0))

plt.tight_layout()

import os
os.makedirs("fig", exist_ok=True)
plt.savefig("fig/Cp_vs_T.pdf", dpi=150)
plt.savefig("fig/Cp_vs_T.png", dpi=150)
print(f"Figures sauvegardees : fig/Cp_vs_T.pdf / .png")
print(f" cart CoolProp vs. GP a 1100 K, 1 bar : +{ecart:.1f} %")
plt.show()
```

A18 — gpcoolprop.py (1/5)

Comparatif de rendement G.P. vs gaz reel

```
"""
Graphe comparatif : Ericsson gaz parfait vs CoolProp
Deux sous-graphes c t e a c t e :
- Gauche : eta en fonction de Tmax (rp et Tmin fixes)
- Droite : eta en fonction de rp (Tmax et Tmin fixes)
"""

import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker
from CoolProp.CoolProp import PropsSI as CP
from functools import lru_cache

# __ Parametres -----
T_MIN = 300 # K (fixe)
T_MAX = 1100 # K (fixe pour le graphe de droite)
P_MIN = 1e5 # Pa = 1 bar (fixe)
RP_REF = 10 # rapport de pression de reference (graphe gauche)

T_MAX_RANGE = np.linspace(600, 1300, 120) # abscisse gauche
RP_RANGE = np.linspace(1.5, 25, 120) # abscisse droite

FLUID = "Air"

# __ CoolProp avec cache -----
@lru_cache(maxsize=50000)
def get_s(T, P):
    return CP("S", "T", float(T), "P", float(P), FLUID)

# __ Ericsson CoolProp -----
def eta_ericsson_coolprop(T_min, T_max, P_min, rp):
```

A18 — gpcoolprop.py (2/5)

Comparatif de rendement G.P. vs gaz reel

```
s4 = get_s(T_max, P_min)
Q_in  = T_max * (s4 - s3)
Q_out = T_min * (s1 - s2)
W_net = Q_in - Q_out
return W_net / Q_in

# __ Ericsson gaz parfait __-----
# eta_GP = 1 - Tmin/Tmax (independant de rp avec regeneration)
def eta_ericsson_gp(T_min, T_max):
    return 1.0 - T_min / T_max

# __ Calcul des courbes __-----
# Graphe gauche : f(Tmax), rp fixe
eta_cp_tmax = [eta_ericsson_coolprop(T_MIN, T, P_MIN, RP_REF) * 100
               for T in T_MAX_RANGE]
eta_gp_tmax = [eta_ericsson_gp(T_MIN, T) * 100
               for T in T_MAX_RANGE]
eta_carnot_tmax = [eta_ericsson_gp(T_MIN, T) * 100    # = Carnot
                  for T in T_MAX_RANGE]

# Graphe droit : f(rp), Tmax fixe
# Pour gaz parfait : eta independant de rp -> ligne horizontale
eta_cp_rp = [eta_ericsson_coolprop(T_MIN, T_MAX, P_MIN, rp) * 100
             for rp in RP_RANGE]
eta_gp_rp = [eta_ericsson_gp(T_MIN, T_MAX) * 100] * len(RP_RANGE)

# __ Mise en page __-----
BLEU      = "#1A3A6B"    # bleuMarine
ROUGE     = "#C0392B"    # rougeCycle
ORANGE    = "#D36900"    # orangeAcc
GRIS      = "#888888"
```

A18 — gpcoolprop.py (3/5)

Comparatif de rendement G.P. vs gaz reel

```
# __ Sous-graphe gauche -----
ax = axes[0]
ax.plot(T_MAX_RANGE, eta_cp_tmax,
        color=BLEU, lw=2.2, label="Ericsson - CoolProp (gaz reel)")
ax.plot(T_MAX_RANGE, eta_gp_tmax,
        color=ROUGE, lw=2.2, linestyle="--",
        label="Ericsson - Gaz parfait ( $\eta = 1 - T_{\min}/T_{\max}$ )")

# Annotation de l'ecart a Tmax = 1100 K
T_ref = 1100
eta_cp_ref = eta_ericsson_coolprop(T_MIN, T_ref, P_MIN, RP_REF) * 100
eta_gp_ref = eta_ericsson_gp(T_MIN, T_ref) * 100
ecart = eta_gp_ref - eta_cp_ref
ax.annotate("",
            xy=(T_ref, eta_cp_ref), xytext=(T_ref, eta_gp_ref),
            arrowprops=dict(arrowstyle="<->", color=ORANGE, lw=1.8))
ax.text(T_ref + 18, (eta_cp_ref + eta_gp_ref) / 2,
        f"$\\Delta\\eta = {ecart:.1f}$ pts",
        color=ORANGE, fontsize=9, va="center")

ax.set_xlabel("$T_{\max}$ (K)", fontsize=11)
ax.set_ylabel("$\\eta$ (%)", fontsize=11)
ax.set_title(f"Influence de $T_{\max}$\n"
            f"$T_{\min}={T_MIN}\\$, \\mathrm{{K}}$, "
            f"$r_p={RP_REF}$",
            fontsize=10)
ax.set_xlim(T_MAX_RANGE[0], T_MAX_RANGE[-1])
ax.set_ylim(45, 82)
ax.xaxis.set_minor_locator(ticker.AutoMinorLocator())
ax.yaxis.set_minor_locator(ticker.AutoMinorLocator())
```

A18 — gpcoolprop.py (4/5)

Comparatif de rendement G.P. vs gaz reel

```
# __ Sous-graphe droit -----
ax = axes[1]
ax.plot(RP_RANGE, eta_cp_rp,
        color=BLEU, lw=2.2, label="Ericsson - CoolProp (gaz reel)")
ax.plot(RP_RANGE, eta_gp_rp,
        color=ROUGE, lw=2.2, linestyle="--",
        label="Ericsson - Gaz parfait (constant)")

# Annotation : eta GP est constant car independant de rp
ax.text(RP_RANGE[-1] - 1.5, eta_gp_rp[0] + 0.5,
        "GP :  $\eta$  indep. de  $r_p$ ",
        color=ROUGE, fontsize=8.5, ha="right")

# Annotation ecart a rp = 10
rp_ref_idx = np.argmin(np.abs(RP_RANGE - RP_REF))
eta_cp_rp10 = eta_cp_rp[rp_ref_idx]
eta_gp_rp10 = eta_gp_rp[rp_ref_idx]
ecart_rp = eta_gp_rp10 - eta_cp_rp10
ax.annotate("",
            xy=(RP_REF, eta_cp_rp10), xytext=(RP_REF, eta_gp_rp10),
            arrowprops=dict(arrowstyle="<->", color=ORANGE, lw=1.8))
ax.text(RP_REF + 0.6, (eta_cp_rp10 + eta_gp_rp10) / 2,
        f" $\Delta\eta = \text{ecart\_rp} \cdot 10^{\text{pts} \cdot (\text{RP\_REF})}$ ",
        color=ORANGE, fontsize=9, va="center")

ax.set_xlabel(" $r_p = P_{\text{max}}/P_{\text{min}}$ ", fontsize=11)
ax.set_ylabel(" $\eta$  (%)", fontsize=11)
ax.set_title(f"Influence de  $r_p$ \n"
            f" $T_{\text{min}} = T_{\text{MIN}}$ ,  $\mathrm{{K}}$ , "\n"
            f" $T_{\text{max}} = T_{\text{MAX}}$ ,  $\mathrm{{K}}$ ",
            fontsize=10)
```

A18 — gpcoolprop.py (5/5)

Comparatif de rendement G.P. vs gaz reel

```
ax.grid(True, which="major", linestyle="--", alpha=0.4)
ax.grid(True, which="minor", linestyle=":", alpha=0.2)
ax.legend(fontsize=8.5, loc="lower right", framealpha=0.9)

# __ Titre global -----
fig.suptitle(
    "Ericsson : comparaison gaz parfait vs gaz reel (CoolProp)\n"
    r"$P_{\min} = 1, \mathrm{\bar{}}$, fluide : Air",
    fontsize=11, y=0.98
)

plt.savefig("gp_vs_coolprop.pdf", dpi=300, bbox_inches="tight")
plt.savefig("gp_vs_coolprop.png", dpi=200, bbox_inches="tight")
print("Figures sauvegardees : gp_vs_coolprop.pdf / .png")

# __ Affichage des valeurs cles -----
print(f"\nValeurs de reference (Tmax={T_ref} K, rp={RP_REF}) : ")
print(f" eta gaz parfait = {eta_gp_ref:.2f} %")
print(f" eta CoolProp = {eta_cp_ref:.2f} %")
print(f" cart = {cart:.2f} points")
```

A19 — generate *figures.py*(1/8)

Automatisation des graphes

```
"""
Generation des figures pour la presentation TIPE.
Utilise les modules existants src/Brayton.py, src/Ericsson.py, src/Stirling.py.
"""
import sys, os
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

os.makedirs("fig", exist_ok=True)

# __ Palette identique a la presentation beamer -----
C_B = (192/255, 57/255, 43/255) # rougeCycle
C_E = ( 26/255, 58/255, 107/255) # bleuMarine
C_S = ( 30/255, 132/255, 73/255) # vertCycle
C_A = (211/255, 105/255, 0/255) # orangeAcc
C_D = ( 50/255, 50/255, 60/255) # grisTexte

plt.rcParams.update({
    "font.family": "serif", "font.size": 8,
    "axes.titlesize": 8, "axes.labelsize": 7.5,
    "xtick.labelsize": 7, "ytick.labelsize": 7,
    "legend.fontsize": 6.5, "lines.linewidth": 1.6,
    "axes.grid": True, "grid.alpha": 0.3, "figure.dpi": 150,
})
SEG_C = [C_B, C_E, C_S, C_A]

def save(name):
```

A19 — generate *figures.py* (2/8)

Automatisation des graphes

```
# =====
# 1. Importer et calculer les 3 cycles
# =====
from src import Brayton, Ericsson, Stirling

TMIN, TMAX = 300.0, 1100.0
PMIN, PMAX = 2e5, 20e5
VMIN, VMAX = 0.3, 1.0
RP = 10

print("Calcul des cycles (peut prendre ~30 s)...")
rb = Brayton.cycle_brayton(T1=TMIN, P1=PMIN, pressure_ratio=RP, T3=TMAX, show_plot=False)
re = Ericsson.cycle_ericsson(T_min=TMIN, T_max=TMAX, P_min=PMIN, P_max=PMAX, show_plot=False)
rs = Stirling.cycle_stirling(T_min=TMIN, T_max=TMAX, V_min=VMIN, V_max=VMAX, show_plot=False)
print("Cycles calculés.")

LB = ["1->2 Isentropique", "2->3 Isobare", "3->4 Isentropique", "4->1 Isobare"]
LE = ["1->2 Isotherme", "2->3 Isobare", "3->4 Isotherme", "4->1 Isobare"]
LS = ["1->2 Isotherme", "2->3 Isochore", "3->4 Isotherme", "4->1 Isochore"]

SEGS_PV = [("V12", "P12"), ("V23", "P23"), ("V34", "P34"), ("V41", "P41")]
SEGS_TS = [("s12", "T12"), ("s23", "T23"), ("s34", "T34"), ("s41", "T41")]
T_CORNERS = [TMIN, TMIN, TMAX, TMAX]

def pv_ts(res, title, col, labels, prefix):
    cv = res["curves"]
    # - P-V -
    fig, ax = plt.subplots(figsize=(3.0, 2.1))
    for (vk, pk), lb, c in zip(SEGS_PV, labels, SEG_C):
        ax.plot(cv[vk], np.array(cv[pk])/1e5, color=c, label=lb)
```

A19 — generate *figures.py* (3/8)

Automatisation des graphes

```

ax.set_xlabel("Volume spec. (m3/kg)"); ax.set_ylabel("Pression (bar)")
ax.set_title(f"Cycle {title} - P-V", color=col, fontweight="bold")
ax.legend(loc="upper right", framealpha=0.8, fontsize=5.5)
save(f"{prefix}_PV.pdf")

# - T-s -
fig, ax = plt.subplots(figsize=(3.0, 2.1))
for (sk,tk), lb, c in zip(SEGS_TS, labels, SEG_C):
    ax.plot(cv[sk], cv[tk], color=c, label=lb)
ax.scatter(res["s"], T_CORNERS, s=18, color=C_D, zorder=5)
for i,(s,t) in enumerate(zip(res["s"], T_CORNERS)):
    ax.annotate(str(i+1),(s,t), xytext=(3,3), textcoords="offset points",
                fontsize=6, color=C_D)
ax.set_xlabel("Entropie (J/kg/K)"); ax.set_ylabel("Temperature (K)")
ax.set_title(f"Cycle {title} - T-s", color=col, fontweight="bold")
ax.legend(loc="upper left", framealpha=0.8, fontsize=5.5)
save(f"{prefix}_Ts.pdf")

print("Diagrammes P-V / T-s...")
pv_ts(rb, "Brayton", C_B, LB, "brayton")
pv_ts(re, "Ericsson", C_E, LE, "ericsson")
pv_ts(rs, "Stirling", C_S, LS, "stirling")

# =====
# 2. Gaz parfait vs CoolProp : eta(Tmax)
# =====
print("Gaz parfait vs CoolProp...")
Tmaxs = np.linspace(600, 1300, 28)
gp, cp_e, cp_b = [], [], []
for Tm in Tmaxs:
    gp.append((1 - TMIN/Tm)*100)

```

Automatisation des graphes

```

try:
    r = Brayton.cycle_brayton(T1=TMIN,P1=PMIN,pressure_ratio=RP,T3=float(Tm),show_plot=False)
    cp_b.append(r["energetics"]["eta"]*100)
except: cp_b.append(np.nan)

fig, ax = plt.subplots(figsize=(4.2, 2.8))
ax.plot(Tmaxs, gp, color=C_A, ls="--", label="Ericsson gaz parfait")
ax.plot(Tmaxs, cp_e, color=C_E, label="Ericsson CoolProp")
ax.plot(Tmaxs, cp_b, color=C_B, label=f"Brayton CoolProp ($r_p$={RP})")
ax.set_xlabel("$T_{\\mathrm{Tmax}}$ (K)"); ax.set_ylabel("Rendement (%)")
ax.set_title("Gaz parfait vs gaz reel (CoolProp)"); ax.legend()
save("gp_vs_coolprop.pdf")

# =====
# 3. eta(Tmax) - 3 cycles
# =====
print("eta vs Tmax...")
eta_e, eta_s, eta_b, carnot = [], [], [], []
for Tm in Tmaxs:
    carnot.append((1 - TMIN/Tm)*100)
    try:
        r = Ericsson.cycle_ericsson(T_min=TMIN,T_max=float(Tm),P_min=PMIN,P_max=PMAX,show_plot=False)
        eta_e.append(r["energetics"]["eta"]*100)
    except: eta_e.append(np.nan)
    try:
        r = Stirling.cycle_stirling(T_min=TMIN,T_max=float(Tm),V_min=VMIN,V_max=VMAX,show_plot=False)
        eta_s.append(r["energetics"]["eta"]*100)
    except: eta_s.append(np.nan)
    try:
        r = Brayton.cycle_brayton(T1=TMIN,P1=PMIN,pressure_ratio=RP,T3=float(Tm),show_plot=False)
        eta_b.append(r["energetics"]["eta"]*100)
    except: eta_b.append(np.nan)

```

A19 — generate *figures.py* (5/8)

Automatisation des graphes

```

ax.plot(Tmaxs, eta_e, color=C_E, label="Ericsson")
ax.plot(Tmaxs, eta_s, color=C_S, ls="--", label="Stirling")
ax.plot(Tmaxs, eta_b, color=C_B, label=f"Brayton ($r_p$={RP})")
ax.set_xlabel("$T_{\mathrm{max}}$ (K)"); ax.set_ylabel("Rendement (%)")
ax.set_title("Influence de $T_{\mathrm{max}}$ sur $\eta$"); ax.legend()
save("eta_Tmax.pdf")

# =====
# 4. Wnet(Pmax) - Ericsson
# =====
print("Wnet vs Pmax...")
Pmaxs = np.linspace(5e5, 30e5, 22)
wnet_e = []
for Pm in Pmaxs:
    try:
        r = Ericsson.cycle_ericsson(T_min=TMIN, T_max=TMAX, P_min=PMIN, P_max=float(Pm), show_plot=False)
        wnet_e.append(r["energetics"]["W_cycle"]/1000)
    except: wnet_e.append(np.nan)

fig, ax = plt.subplots(figsize=(3.8, 2.6))
ax.plot(Pmaxs/1e5, wnet_e, color=C_E)
ax.set_xlabel("$P_{\mathrm{max}}$ (bar)"); ax.set_ylabel("$W_{\mathrm{net}}$ (kJ/kg)")
ax.set_title("Ericsson : $W_{\mathrm{net}}$ vs $P_{\mathrm{max}}$")
save("Wnet_Pmax.pdf")

# =====
# 5. eta(rp) - Brayton vs Ericsson
# =====
print("eta vs rp...")
RPs = np.arange(2, 25)
eta_b_rp, eta_e_rp = [], []

```

A19 — generate *figures.py* (6/8)

Automatisation des graphes

```

except: eta_b_rp.append(np.nan)
try:
    r = Ericsson.cycle_ericsson(T_min=TMIN,T_max=TMAX,P_min=PMIN,P_max=PMIN*rp,show_plot=False)
    eta_e_rp.append(r["energetics"]["eta"]*100)
except: eta_e_rp.append(np.nan)

fig, ax = plt.subplots(figsize=(3.8, 2.6))
ax.plot(RPs, eta_b_rp, color=C_B, label="Brayton")
ax.plot(RPs, eta_e_rp, color=C_E, label="Ericsson (constant)")
ax.set_xlabel("Rapport de pression $r_p$"); ax.set_ylabel("Rendement (%)")
ax.set_title("$\\eta$ vs $r_p$ : Brayton / Ericsson"); ax.legend()
save("eta_rp.pdf")

# =====
# 6. eta(Tmin) - Brayton / Ericsson / Stirling
# =====
print("eta vs Tmin...")
Tmins = np.linspace(280, 450, 22)
eta_b_t, eta_e_t, eta_s_t, carnot_t = [], [], [], []
for Tm in Tmins:
    carnot_t.append((1 - Tm/TMAX)*100)
    try:
        r = Brayton.cycle_brayton(Ti=float(Tm),P1=PMIN,pressure_ratio=RP,T3=TMAX,show_plot=False)
        eta_b_t.append(r["energetics"]["eta"]*100)
    except: eta_b_t.append(np.nan)
    try:
        r = Ericsson.cycle_ericsson(T_min=float(Tm),T_max=TMAX,P_min=PMIN,P_max=PMAX,show_plot=False)
        eta_e_t.append(r["energetics"]["eta"]*100)
    except: eta_e_t.append(np.nan)
    try:
        r = Stirling.cycle_stirling(T_min=float(Tm),T_max=TMAX,V_min=VMIN,V_max=VMAX,show_plot=False)

```

Automatisation des graphes

```

ax.plot(Tmins, carnot_t, color=C_D, ls=":", lw=1.2, label="Carnot")
ax.plot(Tmins, eta_b_t, color=C_B, ls="-.", label="Brayton")
ax.plot(Tmins, eta_e_t, color=C_E, label="Ericsson")
ax.plot(Tmins, eta_s_t, color=C_S, ls="--", label="Stirling")
ax.set_xlabel("$T_{\mathrm{min}}$ (K)"); ax.set_ylabel("Rendement (%)")
ax.set_title("Influence de $T_{\mathrm{min}}$ sur $\eta$"); ax.legend()
save("eta_Tmin.pdf")

# __ 6b. Qin(Tmin) - Ericsson / Stirling -----
print("Qin vs Tmin...")
qin_b_t, qin_e_t, qin_s_t = [], [], []
for Tm in Tmins:
    try:
        r = Brayton.cycle_brayton(Ti=float(Tm),Pi=PMIN,pressure_ratio=RP,T3=TMAX,show_plot=False)
        qin_b_t.append(r["energetics"]["Q_in"]/1000)
    except: qin_b_t.append(np.nan)
    try:
        r = Ericsson.cycle_ericsson(T_min=float(Tm),T_max=TMAX,P_min=PMIN,P_max=PMAX,show_plot=False)
        qin_e_t.append(r["energetics"]["Q_in"]/1000)
    except: qin_e_t.append(np.nan)
    try:
        r = Stirling.cycle_stirling(T_min=float(Tm),T_max=TMAX,V_min=VMIN,V_max=VMAX,show_plot=False)
        qin_s_t.append(r["energetics"]["Q_in"]/1000)
    except: qin_s_t.append(np.nan)

fig, ax = plt.subplots(figsize=(4.2, 2.8))
ax.plot(Tmins, qin_b_t, color=C_B, ls="-. ", label="Brayton")
ax.plot(Tmins, qin_e_t, color=C_E, label="Ericsson")
ax.plot(Tmins, qin_s_t, color=C_S, ls="--", label="Stirling")
ax.set_xlabel("$T_{\mathrm{min}}$ (K)"); ax.set_ylabel("$Q_{\mathrm{in}}$ (kJ/kg)")
ax.set_title("Influence de $T_{\mathrm{min}}$ sur $Q_{\mathrm{in}}$"); ax.legend()

```

A19 — generate *figures.py* (8/8)

Automatisation des graphes

```
# =====
print("Comparaisons barres...")
cycles = ["Brayton", "Ericsson", "Stirling"]
colors = [C_B, C_E, C_S]
etas = [rb["energetics"]["eta"]*100,
        re["energetics"]["eta"]*100,
        rs["energetics"]["eta"]*100]
wnets = [rb["energetics"]["W_net"]/1000,
        re["energetics"]["W_cycle"]/1000,
        rs["energetics"]["W_cycle"]/1000]

fig, ax = plt.subplots(figsize=(3.8, 2.6))
bars = ax.bar(cycles, etas, color=colors, width=0.5, edgecolor="white")
for b, v in zip(bars, etas):
    ax.text(b.get_x()+b.get_width()/2, b.get_height()+0.4,
            f"{v:.1f}%", ha="center", fontsize=7, fontweight="bold")
ax.set_ylabel("Rendement (%)"); ax.set_ylim(0, max(etas)*1.18)
ax.set_title("Comparaison  $\eta$  - 3 cycles\n($T_{\min}=300$ K,  $T_{\max}=1100$ K)")
save("comparaison_eta.pdf")

fig, ax = plt.subplots(figsize=(3.8, 2.6))
bars = ax.bar(cycles, wnets, color=colors, width=0.5, edgecolor="white")
for b, v in zip(bars, wnets):
    ax.text(b.get_x()+b.get_width()/2, b.get_height()+1.5,
            f"{v:.0f}", ha="center", fontsize=7, fontweight="bold")
ax.set_ylabel("$W_{\mathrm{net}}$ (kJ/kg)"); ax.set_ylim(0, max(wnets)*1.20)
ax.set_title("Comparaison  $W_{\mathrm{net}}$  - 3 cycles")
save("comparaison_Wnet.pdf")

print("\nToutes les figures generees dans fig/")$ 
```

A20 — calculs.py (1/6)

Traitement des resultats experimentaux

```
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from matplotlib.patches import FancyArrowPatch
import matplotlib.ticker as ticker
from scipy.ndimage import uniform_filter1d

lines = open("exp/mesures_TIPE_pv.txt").readlines()
V_data, P_data = [], []
for line in lines:
    line = line.strip()
    if not line or line.startswith("V") or line.startswith("T"):
        continue
    parts = line.replace(", ", ".").split()
    if len(parts) == 2:
        try:
            V_data.append(float(parts[0])); P_data.append(float(parts[1]))
        except: pass

V = np.array(V_data); P = np.array(P_data)

# Separer les deux branches par mediane de P
P_med = np.median(P)
mask_haut = P > P_med; mask_bas = P <= P_med
V_h = V[mask_haut]; P_h = P[mask_haut]
V_b = V[mask_bas]; P_b = P[mask_bas]

# Trier
idx_h = np.argsort(V_h); idx_b = np.argsort(V_b)
V_h = V_h[idx_h]; P_h = P_h[idx_h]
```

A20 — calculs.py (2/6)

Traitement des resultats experimentaux

```
W_bas = np.trapezoid(P_b, V_b)
W_net_mbar_cm3 = W_haut - W_bas
W_net_J = W_net_mbar_cm3 * 100 * 1e-6 # mbar*cm3 -> J

print(f"=== Resultats experimentaux ===")
print(f"V_min = {V.min():.3f} cm3 V_max = {V.max():.3f} cm3")
print(f"r_v = {V.max()/V.min():.4f} (constructeur = 1.375)")
print(f"P_min = {P.min():.1f} mbar P_max = {P.max():.1f} mbar")
print(f"P_moy_haut = {P_h.mean():.1f} mbar P_moy_bas = {P_b.mean():.1f} mbar")
print(f"P_moyen = {P_h.mean()-P_b.mean():.1f} mbar")
print()
print(f"W_net = {W_net_mbar_cm3:.2f} mbar/cm3 = {W_net_J*1000:.3f} mJ = {W_net_J:.6f} J")
print()

# Temperature
T_dem = 124 + 273.15 # K
T_max = 228 + 273.15 # K
T_amb = 20 + 273.15 # K (T_froid supposee ambiante)
print(f"T_demarrage = {T_dem:.1f} K T_max = {T_max:.1f} K")
print(f"eta_Carnot (T_amb=293K, T_max=501K) = {(1-293/T_max)*100:.2f}%")
print(f"eta_Carnot (T_dem=397K, T_max=501K) = {(1-T_dem/T_max)*100:.2f}%")
print()

# Puissance
for n in [700, 900, 1000]:
    f = n/60
    P_meca = W_net_J * f
    print(f"n = {n} tr/min -> P_meca = {P_meca*1000:.2f} mW")
```

A20 — calculs.py (3/6)

Traitement des resultats experimentaux

```

r_air = 287 # J/kg/K
V_moy_m3 = ((V.min()+V.max())/2) * 1e-6
P_moy_Pa = ((P.min()+P.max())/2) * 100
T_moy_K = (T_dem + T_max) / 2
m_air = P_moy_Pa * V_moy_m3 / (r_air * T_moy_K)
print(f"\nMasse d'air = {m_air*1e6:.2f} ug")
print(f"W_net/kg = {W_net_J/m_air:.1f} J/kg")

# __ Donnees -----
lines = open("exp/mesures_TIPE_pv.txt").readlines()
V_data, P_data = [], []
for line in lines:
    line = line.strip()
    if not line or line.startswith("V") or line.startswith("T"): continue
    parts = line.replace(",",".").split()
    if len(parts) == 2:
        try: V_data.append(float(parts[0])); P_data.append(float(parts[1]))
        except: pass

V = np.array(V_data); P = np.array(P_data)

# Separer branches
P_med = np.median(P)
mask_h = P > P_med; mask_b = P <= P_med
V_h = V[mask_h]; P_h = P[mask_h]
V_b = V[mask_b]; P_b = P[mask_b]
idx_h = np.argsort(V_h); idx_b = np.argsort(V_b)
V_h = V_h[idx_h]; P_h = P_h[idx_h]
  
```

A20 — calculs.py (4/6)

Traitement des resultats experimentaux

```

V_h_sm = V_h; P_h_sm = uniform_filter1d(P_h, N)
V_b_sm = V_b; P_b_sm = uniform_filter1d(P_b, N)

# W_net
W_net_J = np.trapezoid(P_h, V_h)*100*1e-6 - np.trapezoid(P_b, V_b)*100*1e-6

# __ Modele Theorique -----
import sys, os
sys.path.insert(0, os.path.abspath('src'))
from Stirling import cycle_stirling

m_air_kg = m_air
v_min_m3kg = (V.min()*1e-6) / m_air_kg
v_max_m3kg = (V.max()*1e-6) / m_air_kg

# Calculate cycle theoretical
# We use T_dem as T_min
res_th = cycle_stirling(T_min=T_dem, T_max=T_max, V_min=v_min_m3kg, V_max=v_max_m3kg, show_plot=False)

V_th = []
P_th = []
# The cycle is divided into 4 transformations in res_th['curves']
V_th = np.concatenate([res_th['curves']['V12'], res_th['curves']['V23'], res_th['curves']['V34'], res_th['curves']['V41']])
P_th = np.concatenate([res_th['curves']['P12'], res_th['curves']['P23'], res_th['curves']['P34'], res_th['curves']['P41']])

V_th = np.array(V_th) * m_air_kg * 1e6 # convert to cm3
P_th = np.array(P_th) / 100 # convert Pa to mbar

```

A20 — calculs.py (5/6)

Traitement des resultats experimentaux

```
fig, ax = plt.subplots(figsize=(7.5, 5.0))

# Nuage de points bruts
ax.scatter(V_h, P_h, c=ROUGE, s=6, alpha=0.35, zorder=2, label="_nolegend_")
ax.scatter(V_b, P_b, c=BLEU, s=6, alpha=0.35, zorder=2, label="_nolegend_")

# Courbes lissees
ax.plot(V_h_sm, P_h_sm, color=ROUGE, lw=2.2, zorder=4,
        label=r'Detente isotherme ($T_{\rm max}$=228\, degC)')
ax.plot(V_b_sm, P_b_sm, color=BLEU, lw=2.2, zorder=4,
        label=r'Compression isotherme ($T_{\rm min}$=124\, degC demarrage)')

# Overlay theorique
ax.plot(V_th, P_th, color='black', lw=1.5, linestyle='--', zorder=5, label='Modele theorique CoolProp')

# Point remarquable
ax.scatter([V_h_sm[len(V_h_sm)//2]], [P_h_sm[len(P_h_sm)//2]], c='red', s=40, zorder=6, label='Point de
fonctionnement typique')
ax.annotate('Application', xy=(V_h_sm[len(V_h_sm)//2], P_h_sm[len(P_h_sm)//2]), xytext=(V_h_sm[len(V_h_sm)
//2]+1.5, P_h_sm[len(P_h_sm)//2]+30),
          arrowprops=dict(arrowstyle="->", color='red'), color='red')

# Remplissage de l'aire W_net
V_fill = np.concatenate([V_h_sm, V_b_sm[::-1]])
P_fill = np.concatenate([P_h_sm, P_b_sm[::-1]])
ax.fill(V_fill, P_fill, alpha=0.12, color=ORANGE, zorder=1)
ax.text(38.5, 970, f'$W_{\rm net}$ = {W_net_J*1000:.0f} mJ',
        fontsize=10, color=ORANGE, fontweight='bold', ha='center')
```

A20 — calculs.py (6/6)

Traitement des resultats experimentaux

```

bbox=dict(boxstyle='round,pad=0.3', facecolor='white',
          edgecolor=VERT, alpha=0.85))

# Fleche sens de rotation (horaire en PV = moteur)
ax.annotate("", xy=(40, 1040), xytext=(36, 1040),
            arrowprops=dict(arrowstyle="-|>", color=ORANGE,
                            lw=1.8, mutation_scale=16))
ax.text(38.0, 1055, "sens cycle moteur", fontsize=8,
        color=ORANGE, ha='center')

ax.set_xlabel("Volume $V$ (cm3)", fontsize=11)
ax.set_ylabel("Pression $P$ (mbar)", fontsize=11)
ax.set_title(
    "Diagramme P-V mesure - Moteur Stirling Jeulin 204081\n"
    r"ExA0 Foxy - $r_v = V_{\rm max}/V_{\rm min}$ = "
    f"$\{V.max()/V.min():.3f\}$ | 342 points | fluide : Air",
    fontsize=10
)

ax.xaxis.set_minor_locator(ticker.AutoMinorLocator())
ax.yaxis.set_minor_locator(ticker.AutoMinorLocator())
ax.legend(fontsize=8.5, loc='upper right', framealpha=0.92)
ax.set_xlim(31.0, 45.0)
ax.set_ylim(780, 1200)
fig.tight_layout()

fig.savefig("fig/stirling_PV.pdf", dpi=300, bbox_inches="tight")
fig.savefig("fig/stirling_PV.png", dpi=200, bbox_inches="tight")
print("OK - stirling_PV.pdf / .png")
print(f"W_net = {W_net*1000:.1f} mJ")
print(f"r_v mesure = {V.max()/V.min():.4f}")

```

A21 — `plotstirlingetacomp.py`(1/2)

Comparatif η vs $T_{\max}$

```
import sys, os
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from concurrent.futures import ThreadPoolExecutor

sys.path.insert(0, os.path.abspath(os.path.join(os.path.dirname(__file__), '..', 'src')))
from Stirling import cycle_stirling

T_min = 124 + 273.15 # T demarrage constant
T_max_list = np.linspace(T_min + 10, T_min + 300, 30)

v_min_m3kg = 1.0992157849889488
v_max_m3kg = 1.5041697701307097

def compute_eta(T_max):
    try:
        res = cycle_stirling(T_min=T_min, T_max=T_max, V_min=v_min_m3kg, V_max=v_max_m3kg, show_plot=False)
        return res["energetics"]["eta"] * 100, res["energetics"]["eta_carnot"] * 100
    except:
        return np.nan, np.nan

print("Computing etas...")
etas_mod = []
etas_car = []

for T in T_max_list:
    mod, car = compute_eta(T)
    etas_mod.append(mod)
    etas_car.append(car)
```

A21 — `plot_stirling_eta_comp.py(2/2)`

Comparatif η vs T_{max}

```
# Point experimental
# T_max exp = 228
# eta_exp: calculer eta a partir du modele pour T_max=228
eta_228 = compute_eta(228 + 273.15)[0]
ax.scatter([228], [eta_228], c='red', s=50, zorder=5, label='Point Typique Experiment. (228 degC)')

ax.set_xlabel("Temperature de source chaude $T_{\text{max}}$ ( degC)")
ax.set_ylabel("Rendement $\eta$ (%)")
ax.set_title(f"volution de $\eta$ avec $T_{\text{max}}$ pour $T_{\text{min}} = \{T_{\text{min}}-273.15:.0f\}$ degC")
ax.grid(True, linestyle="--", alpha=0.6)
ax.legend()
plt.tight_layout()

os.makedirs("fig", exist_ok=True)
fig.savefig("fig/stirling_eta_comp.pdf", dpi=300)
fig.savefig("fig/stirling_eta_comp.png", dpi=200)
print("Saved fig/stirling_eta_comp.pdf")
```